

Compete and Learn? Understanding Competitive Programming IDE Workflows and Designing Context-Aware AI Assistance to support Learning and Assessment

Omkar Joshi*

Centre for Educational Technology
Indian Institute of Technology
Bombay
Mumbai, Maharashtra, India
omkar.joshi21@gmail.com

Katyani Mehta*

Centre for Educational Technology
Indian Institute of Technology
Bombay
Mumbai, Maharashtra, India
katyani201mehta@gmail.com

Samarth Kashyap*

Centre for Educational Technology
Indian Institute of Technology
Bombay
Mumbai, Maharashtra, India
25d1203@iitb.ac.in

Ayushi Purohit*

Centre for Educational Technology
Indian Institute of Technology
Bombay
Mumbai, Maharashtra, India
25m1021@iitb.ac.in

Syaamantak Das*

Centre for Educational Technology
Indian Institute of Technology
Bombay
Mumbai, Maharashtra, India
syaamantak.das@iitb.ac.in

Abstract

Competitive programming platforms prioritize rapid submission and automated judging, but their built-in IDEs often hinder learning-oriented workflows and lack principled AI integration. In preliminary research with competitive programmers (N=74), we found three recurring challenges: workflow fragmentation between local and platform editors, inadequate debugging and test-case visibility, and context-dependent expectations for AI assistance that differ sharply between contest and learning scenarios. Drawing on these observations and HCI principles of cognitive load reduction and scaffolded learning, we present a conceptual IDE design that uses mode-based interaction constraints to govern AI assistance across contexts. Our design separates Contest and Learning modes, introduces inline test-case visualization and error localization, and constrains AI behavior to preserve assessment integrity while advancing pedagogical goals. We contribute empirical evidence of workflow breakdowns in competitive programming environments, design goals for addressing these challenges via interaction design, and a mode-based interaction model for context-appropriate AI assistance in dual-purpose educational platforms.

CCS Concepts

• **Human-centered computing** → **Interaction design; Empirical studies in HCI; HCI theory, concepts and models**; • **Applied computing** → *Interactive learning environments*.

*All authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

CHI EA '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2281-3/26/04

<https://doi.org/10.1145/3772363.3798880>

Keywords

Competitive Programming, Human Computer Interaction, Human-AI Interaction, AI Coding Assistants, Programming Education, Integrated Development Environments

ACM Reference Format:

Omkar Joshi, Katyani Mehta, Samarth Kashyap, Ayushi Purohit, and Syaamantak Das. 2026. Compete and Learn? Understanding Competitive Programming IDE Workflows and Designing Context-Aware AI Assistance to support Learning and Assessment. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798880>

1 Motivation and Problem Context

Millions of programmers worldwide[6] use competitive programming platforms like Codeforces¹ and LeetCode² to compete in algorithmic skills based programming contests. What began as venues for timed coding contests has evolved into essential infrastructure for computer science education, with universities assigning competitive programming (CP) problems in courses[8],[13], job seekers practicing for technical interviews[5],[4], and self-directed learners using these platforms as their primary means of developing problem-solving abilities[9]. For many students, CP represents their most sustained engagement with algorithmic thinking[24],[25].

Yet the integrated development environments these platforms remain designed primarily for rapid code submission rather than learning support. The work of [9] note that these web-based environments are “informal” and “open-ended”, lacking predefined curricula and scaffolding critical for skill development. A recent work [17] directly critique existing platforms, arguing they rely on “user-driven problem selection” instead of personalized, adaptive learning approaches. Built-in IDEs prioritize speed and quick evaluation but typically lack rich debugging tools, informative error messages, or support for iterative testing. This design made

¹<https://codeforces.com/blog/entry/138982>

²<https://grokkipedia.com/page/LeetCode>

sense when CP was purely assessment oriented, but the educational transformation of these platforms means learners now spend far more time practicing and learning than competing. The work of [5] analyzed over 300,000 LeetCode user profiles, suggesting significant platform usage for interview preparation. Thus, the old design of CP IDEs no longer matches the dominant use case. Research on programming environments demonstrates that IDE design profoundly shapes learning. An empirical work by [20] showed that professional IDEs negatively affect first-year students' learning, leading them to develop Incre-Idle with a simpler interface. Their evaluation revealed 66.7% of students could complete tasks easily, and 100% found the GUI intuitive. An earlier work by [15] emphasized the need for learning environments that reduce complexity and make programming more accessible. These studies demonstrate that IDE design profoundly shapes learning outcomes in educational contexts, yet CP platforms, despite serving similar educational functions, have received minimal attention from HCI researchers focused on programming environments. Perhaps because they were viewed as assessment tools rather than learning environments. This gap matters because millions of learners depend on these platforms for education. The inadequacy of built-in IDEs creates fragmented workflows, as we document in our preliminary research. Programmers often write code in external tools like Visual Studio Code, copy it to the submission interface, encounter errors, return to their local environment to debug, and consult documentation or AI tools across browser tabs.[11]. Each transition likely imposes cognitive costs including resumption lag, working memory overhead, and disrupted problem-solving flow. These costs compound under time pressure when learners simultaneously reason about algorithms, manage tool logistics, and cope with deadline stress.

Specific AI coding assistants like GitHub Copilot³ and Tabnine⁴ add new complexity. Many learners use AI to explain errors or clarify concepts when working on CP problems. This assistance can support learning when it provides scaffolding, but undermines skill development when it bypasses cognitive effort. The tension is particularly acute because AI in CP platforms serve dual purposes. During contests, AI threatens assessment validity. During practice, AI might support learning if properly structured. Current platforms and stakeholders⁵[7],[14] are struggling to reach a consensus on whether to prohibit or allow AI use. This situation creates an opportunity to integrate assistance that enhances understanding without simply providing answers.

Addressing this requires reframing CP IDEs as interaction systems that mediate learning, cognition, and responsible AI use. We present a conceptual interaction design demonstrating how integrated tooling and mode-based constraints can reduce fragmentation while governing AI behavior appropriately across contest and learning contexts. We contribute: (1) empirical evidence of workflow fragmentation and context-dependent AI attitudes from seventy-four competitive programmers, (2) design goals for addressing these challenges through integrated tooling and mode-based AI

governance, and (3) a conceptual interaction design demonstrating how these goals might be realized in practice.

2 Preliminary User Research: Participant Details and Methodology for Data Analysis

To understand whether the tensions we observe in CP IDEs reflect broader user experiences, we conducted exploratory research with competitive programmers about their IDE usage patterns and attitudes toward AI assistance between October 2025 and December 2025. We surveyed seventy-four competitive programmers (M=54 (Age range = 18-37), F=20 (Age range = 18-32)), recruited through online communities including Reddit forums, Discord servers (e.g., Codeforces groups, LeetCode study forums) focused on algorithmic problem solving, and university computer science programs. Of those who began the survey, 268 started and 74 completed it (completion rate = 28%). The most commonly used platforms were Codeforces (47%) and LeetCode (45%), both of which rely on browser-based IDEs with limited built-in tooling. Participants included college students (N=53), professional developers (N=17), a small number of educators (N=2), and two individuals in other roles, reflecting the diverse spectrum of users engaging with CP platforms. Experience with CP ranged from (i) less than one year (N=16), (ii) 1-2 years (N=45), (iii) 2-5 years (N=11), and (iv) more than 5 years (N=2), with the modal category being one to two years of experience. While our sample skews toward intermediate practitioners, we achieved representation across the novice-to-experienced spectrum. The survey examined CP IDE usage during contests and practice, workflow challenges, and attitudes toward AI coding assistants in different contexts. The study received ethics approval from our institutional review board. Participation was voluntary; no monetary compensation was provided. For quantitative data, we computed frequencies and percentages for categorical responses and examined distributions across experience levels and roles. Where sample sizes permitted, we conducted chi-square tests to examine whether patterns differed significantly by participant characteristics. For open-ended responses, five researchers independently conducted thematic analysis following established procedures. We iteratively refined themes through multiple readings of responses, discussing interpretations until reaching consensus on the coding scheme and its application to the data. Disagreements were resolved through discussion. We report themes that appeared in responses from at least twenty percent of participants unless otherwise noted.

2.1 Results and Key Observations

Our exploratory research revealed three patterns that motivated the subsequent design work. First, external IDE reliance appears pervasive across experience levels. Seventy percent of participants reported frequently using external tools like Visual Studio Code or IntelliJ⁶ rather than built-in platform IDEs, with eighty-seven percent perceiving significant advantages in doing so. Participants described external IDEs as providing (i) better debugging support, (ii) clearer error messages, and (iii) faster iteration cycles. This suggests built-in environments serve primarily as submission interfaces rather than primary workspaces. Second, eighty-two percent

³<https://github.com/features/copilot>

⁴<https://www.tabnine.com/>

⁵https://www.reddit.com/r/singularity/comments/1il0nb9/openai_claims_their_internal_model_is_top_50_in/

⁶<https://www.jetbrains.com/idea/>

participants consistently described fragmented workflows involving repeated context switching between tools - creates cognitive and temporal costs. The typical pattern involved writing code locally, copying it to the platform for testing, returning to the local environment when errors occurred, consulting documentation across browser tabs, and sometimes seeking help from forums or AI tools before attempting submission again. Participants characterized this fragmentation as creating cognitive overhead and disrupting problem-solving flow, particularly under the time pressure of contests. This workflow breakdown appeared across experience levels, suggesting it reflects structural limitations of current platform designs rather than novice difficulties that resolve with expertise. Third, and most relevant to our design exploration, participants exhibited sharply context-dependent attitudes toward AI assistance. When asked about AI integration during contests, opinions were divided, with approximately 53% viewing AI as potentially valuable, but 32% percent expressing concerns about fairness and assessment validity. In contrast, 55% supported AI assistance in learning and practice contexts, emphasizing desire for help in understanding errors, clarifying concepts, and receiving hints without complete solutions being provided. This context-dependent pattern suggests that uniform approaches to AI integration are unlikely to meet users' nuanced expectations about appropriate assistance in different scenarios.

2.2 Positioning the Design Exploration:

These observations, while preliminary rather than definitive, align with broader research on programming environments and learning support, suggesting they reflect genuine tensions in how CP platforms currently function. More importantly, these observations point to interesting design questions. *If workflows are fragmented and programmers want context-sensitive AI support, how might platform IDEs be redesigned to address these needs through interaction design?* This question motivated the design exploration we present in subsequent sections.

3 From Observations to Design Goals

The patterns we observed in our preliminary research point to opportunities for rethinking CP IDE design through interaction-level interventions. We use these observations as starting points for design reasoning informed by HCI principles, articulating goals that guided our conceptual IDE design rather than claiming validated requirements. Our first design goal addresses workflow fragmentation. Research on interruption and multitasking shows that context switches impose resumption costs where programmers must rebuild mental models of their problem-solving state [16], [1]. Cognitive load theory [22] identifies this as extraneous cognitive load that reduces resources available for algorithmic reasoning. This suggests providing an integrated workspace supporting coding, testing, debugging, and assistance within a single environment to minimize context switches and eliminate parallel tool management. Our second goal addresses error comprehension and fault localization. The HCI principle of visibility of system status [18] suggests systems should provide feedback about what is happening. For programming errors specifically, research shows novices struggle to map abstract error messages to concrete code problems [12],[3]. This

suggests providing debugging support with inline error localization, explanatory feedback about causes, and execution state visibility that supports diagnosis without interrupting problem-solving flow. Our third goal addresses iterative testing workflows. The concept of external cognition in HCI [21] recognizes that programmers use structured test cases as cognitive scaffolding for debugging reasoning. This suggests supporting rapid hypothesis-test cycles through organized test case management, side-by-side input-output visualization, and quick iteration between formulating hypotheses and testing them. Our fourth goal addresses context-appropriate AI assistance. Research on human-AI interaction emphasizes transparency and user control, particularly when AI behavior adapts to context[2],[10]. For educational technologies, scaffolding theory distinguishes support that develops competence from support that substitutes for it [23], [19]. The context-dependent attitudes suggest that AI assistance should be guided by explicit interaction mechanisms that align behavior based on whether programmers are being assessed or learning, with clear visibility into contextual constraints.

4 Conceptual Design Exploration

4.1 Integrated Workspace Design

This integration (Fig. 1 shows the prototype developed (DevPath.ai)) which addresses workflow fragmentation by reducing cognitive costs of context switching. By keeping all essential activities within a single workspace, programmers can maintain continuous mental models of their problem-solving state rather than rebuilding context after each tool transition. The design challenge is achieving integration without overwhelming complexity, requiring progressive disclosure of detail based on programmer needs.

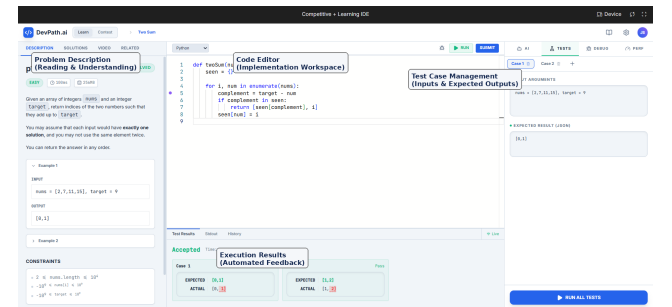


Figure 1: The proposed IDE interface co-locates problem description (left), code editor (center-top), and test case management (right) with results (bottom) within a single workspace. Key features include structured test case organization with labeled cases for rapid switching, side-by-side visibility of inputs and corresponding outputs to support debugging reasoning, and embedded AI assistance accessible without leaving the coding context. This spatial organization addresses workflow fragmentation by eliminating context switches between separate tools for problem reading, coding, testing, and seeking help.

4.2 Contrasting Error Feedback Approaches.

This approach (Fig. 2 shows the prototype) transforms error feedback from a reporting mechanism into a diagnostic support system. Rather than forcing programmers to engage in detective work mapping abstract errors to code locations, the design makes fault localization explicit and provides explanatory context that supports understanding. This aligns with HCI principles of error prevention and recovery while addressing the inadequate debugging support that eighty-two percent participants in our survey described as a primary workflow breakdown.

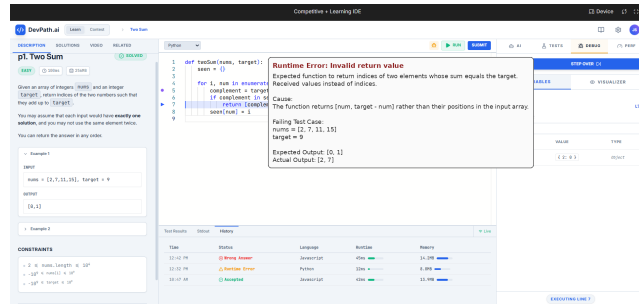


Figure 2: The proposed design integrates error feedback through inline highlighting of problematic code regions with explanatory annotations that describe what went wrong, why it occurred, and which test case triggered the failure. This contextual presentation supports rapid fault localization and error comprehension without disrupting problem-solving flow.

4.3 Mode-Based AI Governance

The mode-based approach (Fig. 3) reflects our finding that fifty-three percent of survey participants viewed AI as potentially valuable during contests while fifty-five percent supported richer AI assistance in learning contexts. By making mode boundaries explicit through interface indicators and capability affordances, the design acknowledges these different expectations while providing transparent control over when different types of assistance are available. The key implementation challenge involves enforcing these constraints in ways that are meaningful rather than trivially circumventable, likely requiring platform-level controls beyond interface design alone.

4.4 Design Implications

This design exploration generates several insights relevant to CP platforms and educational technology more broadly. First, it demonstrates that integrated workflows, enhanced error support, and governed AI assistance can coexist conceptually within a single system rather than requiring separate tools for different purposes. This addresses the fragmentation that currently forces programmers to distribute work across disconnected environments. Second, the mode-based approach suggests a tractable strategy for platforms serving dual assessment and learning purposes. Uniform AI availability creates tensions between supporting learning and preserving assessment validity, while complete prohibition becomes

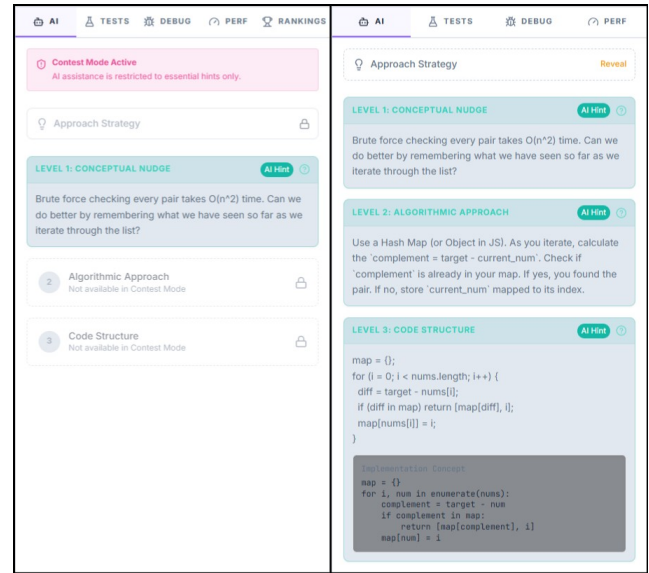


Figure 3: The interface appearance and AI capabilities differ between Contest Mode (left) and Learning Mode (right). Contest Mode displays clear mode indicators, constraints AI assistance to high-level conceptual hints without solution generation, and disables code-completion features to preserve assessment integrity. Learning Mode enables richer AI scaffolding including step-by-step explanations, execution tracing, and reflective prompts while maintaining emphasis on understanding rather than solution provision. Mode boundaries are made explicit through visual indicators and capability affordances, addressing context-dependent expectations for appropriate AI assistance.

impractical as AI tools become ubiquitous. Mode-based governance offers a middle path where AI assistance adapts to context through transparent, user-controlled mechanisms. Third, the exploration reveals that responsibly integrating AI into high-stakes educational environments is not just an interface design problem but also requires platform architecture and policy considerations. Finally, the design demonstrates that competitive programming platforms can be reconceptualized as interaction systems mediating learning, cognition, and ethical AI use rather than merely as code execution and submission tools.

5 Limitations and Future Work

This work has limitations pointing toward future research. Our user research captured self-reported practices rather than observed behavior, and our modest sample limits generalizability. The conceptual design remains unimplemented and unevaluated. Natural next steps include prototyping the design for formative evaluation with competitive programmers and investigating technical approaches to enforcing mode boundaries at the platform level.

6 LLM Disclosure:

No Large Language Model was used to generate substantive content in this manuscript beyond minor copy-editing. During the preparation of this work, the author(s) used Grammarly in order to: Grammar and spelling check. After using these tool(s)/service(s), the author(s) reviewed and edited the content as needed and take(s) full responsibility for the publication's content.

References

- [1] Erik M Altmann and J Gregory Trafton. 2002. Memory for goals: An activation-based model. *Cognitive science* 26, 1 (2002), 39–83.
- [2] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N Bennett, Kori Inkpen, et al. 2019. Guidelines for human-AI interaction. In *Proceedings of the 2019 chi conference on human factors in computing systems*. 1–13.
- [3] Brett A Becker, Kyle Goslin, and Graham Glanville. 2018. The effects of enhanced compiler error messages on a syntax error debugging test. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education*. 640–645.
- [4] Brian Bell, Teresa Thomas, Sang Won Lee, and Chris Brown. 2025. How do software engineering candidates prepare for technical interviews?. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 883–894.
- [5] Jialin Cui, Runqiu Zhang, Fangtong Zhou, Ruochi Li, Yang Song, and Ed Gehringer. 2024. How much effort do you need to expend on a technical interview? a study of leetcode problem solving statistics. In *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 1–10.
- [6] Debeshee Das, Noble Saji Mathews, and Sridhar Chimalakonda. 2022. Exploring security vulnerabilities in competitive programming: An empirical study. In *Proceedings of the 26th International Conference on Evaluation and Assessment in Software Engineering*. 110–119.
- [7] Ahmed El-Kishky, Alexander Wei, Andre Saraiva, Borys Minaiev, Daniel Selsam, David Dohan, Francis Song, Hunter Lightman, Ignasi Clavera, Jakub Pachocki, et al. 2025. Competitive programming with large reasoning models. *arXiv preprint arXiv:2502.06807* (2025).
- [8] Luis H González Guerra, Gilberto Huesca Juárez, Claudia V Pérez-Lezama, Elda G Quiroga González, Gabriela A Campos-García, and Mónica Larre Bolaños Cacho. 2024. Enhancing problem-solving skills: The synergy of competitive programming and gamification strategy. In *2024 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 1–7.
- [9] Carl Christopher Haynes-Magyar and Nathaniel James Haynes-Magyar. 2022. Codespec: A Computer Programming Practice Environment. In *Proceedings of the 2022 ACM Conference on International Computing Education Research-Volume 2*. 32–34.
- [10] Eric Horvitz. 1999. Principles of mixed-initiative user interfaces. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 159–166.
- [11] Omkar Joshi, Kushal Mundra, Deepika Yadav, and Syaamantak Das. 2025. AI Coding Assistants in Competitive Programming: Empirical Studies from India on Human-AI Interactions in Learning, Problem Solving, and Curiosity Development Perspectives. In *International Conference on Artificial Intelligence in Education*. Springer, 181–194.
- [12] Amy J Ko, Brad A Myers, Michael J Coblenz, and Htet Htet Aung. 2006. An exploratory study of how developers seek, relate, and collect relevant information during software maintenance tasks. *IEEE Transactions on software engineering* 32, 12 (2006), 971–987.
- [13] En-Shiun Annie Lee, Luki Danukarjanto, Sadia Sharmin, Shou-Yi Hung, Sicong Huang, and Tong Su. 2024. Exploring student motivation in integration of soft skills training within three levels of computer science programs. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 708–714.
- [14] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [15] António Manso, Célio Gonçalves Marques, Paulo Santos, Luís Lopes, and Raquel Guedes. 2019. Algorithmi IDE-Integrated learning environment for the teaching and learning of algorithmics. In *2019 International Symposium on Computers in Education (SIIE)*. IEEE, 1–6.
- [16] Gloria Mark, Daniela Gudith, and Ulrich Klocke. 2008. The cost of interrupted work: more speed and stress. In *Proceedings of the SIGCHI conference on Human Factors in Computing Systems*. 107–110.
- [17] A Mohanraj, M Nithyasarithi, M Mohammed Safil, S Ravena, and M Narthika. 2024. Smart Problem-Solving Using AI Driven Resource Kit. In *2024 International Conference on Emerging Research in Computational Science (ICERCS)*. IEEE, 1–7.
- [18] Don Norman. 2013. *The design of everyday things: Revised and expanded edition*. Basic books.
- [19] Sadhana Puntambekar and Roland Hubscher. 2005. Tools for scaffolding students in a complex learning environment: What have we gained and what have we missed? *Educational psychologist* 40, 1 (2005), 1–12.
- [20] Matias Salinas, Paul Leger, Hiroaki Fukuda, Nicolás Cardozo, Vanessa de J Duarte, and Ismael Figueroa. 2023. Evaluations of Integrated Programming Environment for First-Year Students in Computer Engineering. *J. Univers. Comput. Sci.* 29, 1 (2023), 73–97.
- [21] Mike Scaife and Yvonne Rogers. 1996. External cognition: how do graphical representations work? *International journal of human-computer studies* 45, 2 (1996), 185–213.
- [22] John Sweller, Jeroen JG Van Merriënboer, and Fred GWC Paas. 1998. Cognitive architecture and instructional design. *Educational psychology review* 10, 3 (1998), 251–296.
- [23] David Wood, Jerome S Bruner, and Gail Ross. 1976. The role of tutoring in problem solving. *Journal of child psychology and psychiatry* 17, 2 (1976), 89–100.
- [24] Kevin KF Yuen, Dennis YW Liu, and Hong Va Leong. 2023. Competitive programming in computational thinking and problem solving education. *Computer Applications in Engineering Education* 31, 4 (2023), 850–866.
- [25] Abdullahi Yusuf and Norah Md Noor. 2024. Modeling students' algorithmic thinking growth trajectories in different programming environments: an experimental test of the Matthew and compensatory hypothesis. *Smart Learning Environments* 11, 1 (2024), 38.